

Fiche Technique

EduGest CM — Back-end (Express / Sequelize / Socket.IO)

Généré le 05/07/2026

1. Identité du projet

Nom	edugest-backend
Version	1.0.0
Framework	Express 5.2
ORM	Sequelize 6.37 + MySQL2
Langage	JavaScript (Node.js)
Point d'entrée	server.js
Port	8080
Licence	ISC

2. Stack technique

express	5.2.1	Framework HTTP
sequelize	6.37.8	ORM MySQL
mysql2	3.22.3	Driver MySQL
jsonwebtoken	9.0.3	JWT auth
bcryptjs	3.0.3	Hash mots de passe
socket.io	4.8.3	WebSocket temps réel
multer	2.2.0	Upload fichiers
cors	2.8.6	Cross-Origin
dotenv	17.4.2	Variables d'environnement
nodemon	3.1.14	Dev : live reload

3. Architecture

Architecture 3 tiers : Routes !' Controllers !' Services !' Modèles Sequelize.

- server.js crée le serveur HTTP + Socket.IO, synchronise la base, puis démarre l'écoute.
- app.js configure Express (middleware JSON, CORS) et monte les 25 modules de routes.
- Chaque entité suit le pattern : route !' controller !' service !' model.
- Pas de gestionnaire d'erreur centralisé : chaque controller gère ses propres erreurs.
- Pas de middleware de validation (express-valid dans package.json mais

non utilisé).

4. Structure des dossiers

```
edugest-backend/
  server.js
  # Entrée : HTTP + Socket.IO +
  DB sync

.env
# DB creds, JWT secret, PORT
seed.js
# Script d'amorçage (données
demo)
uploads/

annonces/                                #
Fichiers joints aux annonces

chat/                                    #
Fichiers partagés dans le chat
src/

app.js                                  #
Configuration Express + routes
config/

database.js                             #
Instance Sequelize
middlewares/

auth.js                                 #
verifyToken (JWT)

rbac.js                                 #
autoriser (RBAC par rôle)

models/                                  #
26 modèles Sequelize

index.js                                #
Agrégateur + associations

controllers/                             #
25 contrôleurs

services/                                #
26 services (logique métier)

routes/                                  #
25 fichiers de routes
socket/

handler.js                              #
Gestionnaire d'events Socket.IO
```

5. Base de données

Connexion

```
Sequelize(edugest, edugest,
edugest123, { host: localhost,
port: 3306, dialect: mysql })
  • sync({ force: false }) au
démarrage — création automatique des
tables.
```

- 26 modèles Sequelize couvrant tous les domaines (Admin, Personne, Eleve, Classe, Conversation, etc.).
- Associations définies dans models/index.js (1-N, N-M, 1-1).

6. Sécurité

Authentification (auth.js)

- Middleware verifyToken : extrait le Bearer token du header Authorization.
- Vérification JWT via jwt.verify(token, JWT_SECRET).
- Décode le payload { idPers, username, role } et le place dans req.user.
- Réponses : 401 si token

manquant, 403 si invalide/expiré.

Autorisation (rbac.js)

- Factory autoriser(roles) : retourne un middleware qui vérifie req.user.role.
- 5 rôles : ADMIN, DIRECTEUR, ENSEIGNANT, RESPONSABLE_ADMIN, PARENT.
- Chaque route définit les rôles autorisés (ex: autoriser([ADMIN, DIRECTEUR])).

7. Routes API principales

Authentification

POST	/api/auth/login	Public
GET	/api/auth/me	Token
PATCH	/api/auth/password	Token
POST	/api/auth/logout	Token

Gestion scolaire

GET/POST	/api/elevés	ADMIN/RESP/DIR/ENS/PARENT
GET	/api/elevés/:matricule/notes	ADMIN/RESP/DIR/ENS/PARENT
GET/POST	/api/enseignants	ADMIN/RESP/DIR
GET/POST	/api/classes	ADMIN/RESP/DIR/ENS
GET/POST	/api/cycles	ADMIN/RESP/DIR/ENS
GET/POST	/api/annees	ADMIN/RESP/DIR
GET/POST	/api/trimestres	ADMIN/RESP/DIR
POST	/api/inscriptions	ADMIN/RESP/DIR
POST	/api/presences/appeal	ADMIN/RESP/DIR/ENS
POST	/api/notes	ADMIN/RESP/DIR/ENS
GET/POST	/api/paiements	ADMIN/RESP/DIR/PARENT
GET/POST	/api/discipline	ADMIN/RESP/DIR/ENS
GET/POST	/api/emploi_du_temps	ADMIN/RESP/DIR/ENS
POST	/api/documents	PARENT
GET	/api/bulletins/eleve/:matricule	Tous

Messagerie et temps réel

GET	/api/conversations	Tous
POST	/api/conversations/get-or-create	Tous
GET	/api/conversations/:id/messages	Tous
POST	/api/messages/upload	Tous (fichier)
GET	/api/communicants	Tous
GET	/api/annonces	Tous
POST	/api/annonces	ADMIN/RESP/DIR
GET	/api/notifications/mes-notifications	Tous

GET

/api/dashboard/stats

Tous

8. Socket.IO (temps réel)

Initialisation

Socket.IO est créé dans server.js et attaché au même serveur HTTP qu'Express. CORS configuré pour localhost:5173, 4173.

Middleware d'authentification

- Lit le token depuis socket.handshake.auth.token ou query.token.
- Vérifie le JWT, rejette la connexion si invalide.
- Stocke socket.user avec les données décodées.

Événements

connection (automatique)	Ajoute l'utilisateur à onlineUsers, rejoint ses rooms, broadcast users:online
conversations:join	Rejoint les rooms des conversations spécifiées
messages:send	Sauvegarde le message en DB, émet messages:new + conversations:updated
messages:edit	Met à jour le contenu (fenêtre 15 min), émet messages:edited
messages:delete	Soft-delete (deletedAt), émet messages:deleted
messages:read	Marque les messages comme lus, émet messages:read-status
annonces:create	Crée l'annonce en DB, broadcast annonces:new, crée les notifications
typing:start	Émet typing:update { isTyping: true }
typing:stop	Émet typing:update { isTyping: false }
disconnect (automatique)	Retire de onlineUsers, broadcast users:online

9. Modèles de données (26)

- Admin — Utilisateurs administrateurs (nom, username, password hashé, actif)
- Personne — Base des utilisateurs (nom, prénom, mobile, typePersonne: 1=ENS,2=DIR,3=RESP_ADMIN,4=PARENT, username, password)
- Eleve — Élèves (matricule, nom, prénom, dateNaissance, sexe, langue, photo)
- Enseignant — Lien Personne !' Cours (idPers FK, idCours FK, Actif)
- Classe — Classes (libellé, idCycle FK)
- Cycle — Cycles éducatifs (libellé, description)
- Salle — Salles de classe (libellé, idClasse FK, idTitulaire FK)
- AnneeAcademique — Années académiques (libellé)
- Trimestre — Trimestres (libellé, dateDebut, dateFin, idAnnee FK)
- Cours — Matières (libellé, coefficient)
- Epreuve — Examens/évaluations (idCours FK, type, noteMax, idClasse FK)
- Note — Notes aux épreuves (idEpreuve FK, matricule FK, note, appréciation)
- Frequent — Inscriptions/Présences (idSalle FK, idAcademi FK, matricule FK, commentaire)
- Paiement — Paiements (matricule FK, montant, date, mode, motif, référence)

- **Scolarite** — Frais scolaires (matricule FK, montantTotal, montantPaye, idAnnee FK)
- **Discipline** — Incidents (matricule FK, typeIncident, description, date, points, statut)
- **EmploiDuTemps** — Emplois du temps (idClasse FK, idCours FK, jour, heureDebut, heureFin, salle)
- **Document** — Documents uploadés (matricule FK, idParent FK, type, nomFichier, chemin, statut)
- **Message** — Messages legacy (expéditeur, destinataire, objet, contenu, statut, fichierUrl)
- **Conversation** — Fils de discussion (type: direct)
- **ConversationParticipant** — Participants aux conversations (idConversation FK, idUser, role, lastReadAt)
- **Annonce** — Annonces broadcast (titre, contenu, auteurId, fichierUrl, cibleRoles)
- **Notification** — Notifications (idDestinataire, titre, message, type, lu)

10. Upload de fichiers (Multer)

Documents	uploads/	pdf, jpg, jpeg, png, gif, doc, docx	10 Mo
Messages	uploads/chat/	pdf, jpg, jpeg, png, gif, doc, docx, xls, xlsx, txt	10 Mo
Annonces	uploads/annonces/	pdf, jpg, jpeg, png, gif, doc, docx, xls, xlsx, txt	10 Mo

Les fichiers sont stockés sur le système de fichiers local avec des noms uniques (Date.now()-random.ext). Le dossier uploads/ est servi statiquement via express.static.

11. Configuration (.env)

DB_NAME	edugest	Base MySQL
DB_USER	edugest	Utilisateur MySQL
DB_PASSWORD	edugest123	Mot de passe MySQL
DB_HOST	localhost	Hôte MySQL
DB_PORT	3306	Port MySQL
JWT_SECRET	edugest_secret_key_2024	Clé JWT
PORT	8080	Port serveur

- **JWT_EXPIRES_IN** non défini dans .env ; valeur par défaut : 24h (hardcodée dans auth.service.js).

12. Démarrage

```
$ node server.js # Production
$ npm run dev # Développement (nodemon)
$ node seed.js # Amorcer la base avec des données de démo
```

Au démarrage, le serveur : (1) charge

le .env, (2) authentifie la connexion MySQL, (3) synchronise les modèles (création des tables si absentes), (4) démarre l'écoute sur le port configuré.